

Caching Strategies for High-Traffic APIs

Introduction

Caching is one of the most effective tools available for reducing latency and protecting backend systems from excessive load. At its core, caching means storing a copy of expensive-to-compute or expensive-to-fetch data somewhere faster to access, so that repeated requests for the same information do not have to repeat the full cost of producing it. In a typical web API, this might mean storing the result of a database query, a rendered page fragment, or a computed aggregation in memory rather than recalculating it on every request. The benefit is straightforward: a cache hit can return a response in microseconds instead of the tens or hundreds of milliseconds a database round trip might take. The difficulty is not deciding whether to cache, but deciding what to cache, for how long, and how to keep it correct.

Cache Placement: Where Caches Live

Caches can sit at several layers of a system, and each layer trades off differently between speed, scope, and staleness risk. A client-side cache, such as a browser cache or a mobile app's local store, is the fastest possible option because it avoids the network entirely, but it only benefits that single client and cannot be centrally invalidated. A CDN or edge cache sits closer to users geographically and is excellent for static or semi-static assets, but is a poor fit for per-user or highly dynamic data. An application-level cache, often backed by Redis or Memcached, sits between the application server and the database. This is usually the layer engineers mean when they say "the cache" in a backend context, because it is shared across all instances of a service and can be invalidated centrally. Finally, some systems use an in-process cache inside the application memory itself, which is extremely fast but does not scale across multiple server instances unless paired with a shared invalidation mechanism.

The Cache-Aside Pattern

The most common caching pattern in production systems is cache-aside, sometimes called lazy-loading. In this pattern, the application code is responsible for checking the cache first; on a miss, it fetches from the source of truth (typically a database), stores the result in the cache, and then returns it to the caller. On a hit, it returns the cached value directly and never touches the database. The advantage of cache-aside is that it only caches data that is actually requested, which keeps the cache lean and avoids wasting memory on data nobody

asks for. The disadvantage is that the first request for any given key always pays the full cost of a cache miss, and if many clients request the same uncached key simultaneously, this can cause a "thundering herd" where dozens of requests all hit the database at once for the same data. A common mitigation is to use a short-lived lock or a request-coalescing mechanism so only one request populates the cache while the others wait for that result.

Write-Through and Write-Behind Caching

An alternative to cache-aside is write-through caching, where every write to the database is immediately mirrored into the cache at write time, rather than waiting for a subsequent read to populate it. This guarantees the cache is never stale immediately after a write, at the cost of adding latency to every write operation, since the write is not considered complete until both the database and the cache have been updated. A more aggressive variant, write-behind (or write-back) caching, writes to the cache immediately and defers the database write to happen asynchronously afterward. This minimizes write latency for the caller but introduces a risk window where data exists only in the cache and could be lost if the cache crashes before the deferred write completes. Write-behind is generally reserved for systems where a small amount of data loss on failure is an acceptable trade-off for significantly lower write latency.

Cache Invalidation Strategies

Cache invalidation is widely considered one of the hardest problems in caching, precisely because there is no universally correct answer. Time-based expiration, using a time-to-live (TTL) value, is the simplest approach: a cached entry automatically expires after a fixed duration, after which the next request is treated as a miss and refreshes the value. TTL is easy to reason about but creates an inherent trade-off between staleness (a long TTL means users may see outdated data for longer) and cache efficiency (a short TTL means more cache misses and more load on the source of truth). Event-based invalidation is more precise: whenever the underlying data changes, the application explicitly deletes or updates the corresponding cache entry. This keeps the cache accurate at all times but requires that every code path capable of mutating the data also remembers to invalidate the right cache keys, which becomes error-prone as a system grows and multiple services or code paths can write to the same underlying data.

Stale-While-Revalidate and Soft TTLs

A pattern that balances freshness against latency is stale-while-revalidate. Under this approach, a cache entry has two expiration markers: a soft TTL and a hard TTL. Once the

soft TTL has passed, the cache still serves the stale value to the caller immediately (so the caller experiences no added latency), but simultaneously triggers a background refresh so that the next request gets an updated value. Only if the hard TTL has also passed does the cache treat the entry as a true miss and force the caller to wait for a fresh fetch. This pattern is popular in high-traffic public APIs because it guarantees that no single request is ever blocked on a slow upstream fetch under normal conditions, while still keeping data reasonably fresh in aggregate.

Cache Stampede Protection

A cache stampede, closely related to the thundering herd problem, occurs when a popular cache key expires and a large volume of concurrent requests all attempt to regenerate it at the same time, overwhelming the backend that the cache was meant to protect. Beyond simple locking, teams commonly apply jittered expiration, where the TTL of each cache entry is randomized slightly (for example, plus or minus ten percent) so that a large batch of keys written at the same time does not all expire in the same millisecond. Another mitigation is probabilistic early expiration, where as a cache entry approaches its expiration time, each request has a small and increasing probability of proactively triggering a refresh before the TTL is actually reached, spreading the regeneration cost out over time rather than concentrating it at the exact expiration moment.

Choosing What Not to Cache

Not all data benefits from caching, and caching the wrong things can introduce more problems than it solves. Data that changes on every request, such as a live counter or a real-time balance, gains little from caching and risks serving incorrect values if the cache is not perfectly synchronized. Highly personalized data with low reuse, such as a one-off computed report for a single user, may cost more in cache memory and invalidation complexity than it saves in compute time, since the cache entry is unlikely to ever be requested again. A useful rule of thumb is that caching pays off most clearly when a piece of data is expensive to compute or fetch, requested frequently by multiple callers, and tolerant of at least brief staleness. When any of those three conditions is false, caching should be evaluated carefully rather than applied by default.

Summary and Practical Checklist

Effective caching is less about picking a single "best" strategy and more about matching the right pattern to the right kind of data. Cache-aside remains the default choice for most read-heavy workloads because of its simplicity and its property of only caching what is

actually requested. Write-through and write-behind are worth the added complexity only when write latency or read freshness immediately after a write genuinely matters for the product experience. Invalidation should default to explicit event-based invalidation wherever the write paths are few and well understood, and fall back to TTL-based expiration, ideally with jitter, wherever writes are too distributed across the codebase to reliably invalidate by hand. Finally, any cache sitting in front of a popular key should assume it will eventually expire under load, and should be paired with stampede protection such as locking, jittered TTLs, or probabilistic early refresh, rather than relying on the optimistic assumption that expirations will always be spread out naturally.